

Boa noite a todos! Bom ver vocês fortes e saudáveis!

Para melhorar o rendimento e a qualidade das gravações:

- **1ª Parte da aula: Teoria Direta**
 - Explicação contínua e sem interrupções.
 - Anotem as dúvidas para o bloco de perguntas no final da teoria.
- **Chat Inteligente**
 - Mandem as dúvidas por escrito a qualquer momento.
 - Alunos experientes: fiquem à vontade para ajudar a responder no chat (ajuda muito)!
- **2ª Parte da aula:**
 - Exercícios e aplicação dos conceitos (ajustável conforme a complexidade do dia).

Não se esqueçam de registrar a presença!



Modelo Preditivo

MÓDULO 2 - SEMANA 5 - AULA 1



SCITI.SC

AGENDA

- Inspiração biológica
- Redes Neurais artificiais
- Quais as relações entre IA, Machine Learning e Deep Learning?
- Perceptron
- Limites do ML Clássico

Inspiração biológica



"ENCEPHALON. Into the Brain, 2026

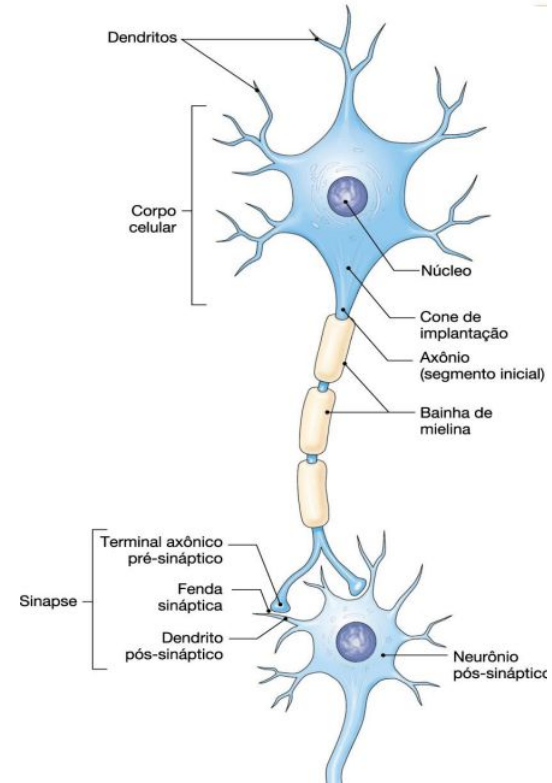


Fonte: The Synapse Revealed (Johnson - 2005).

Inspiração biológica: O cérebro se organiza em redes

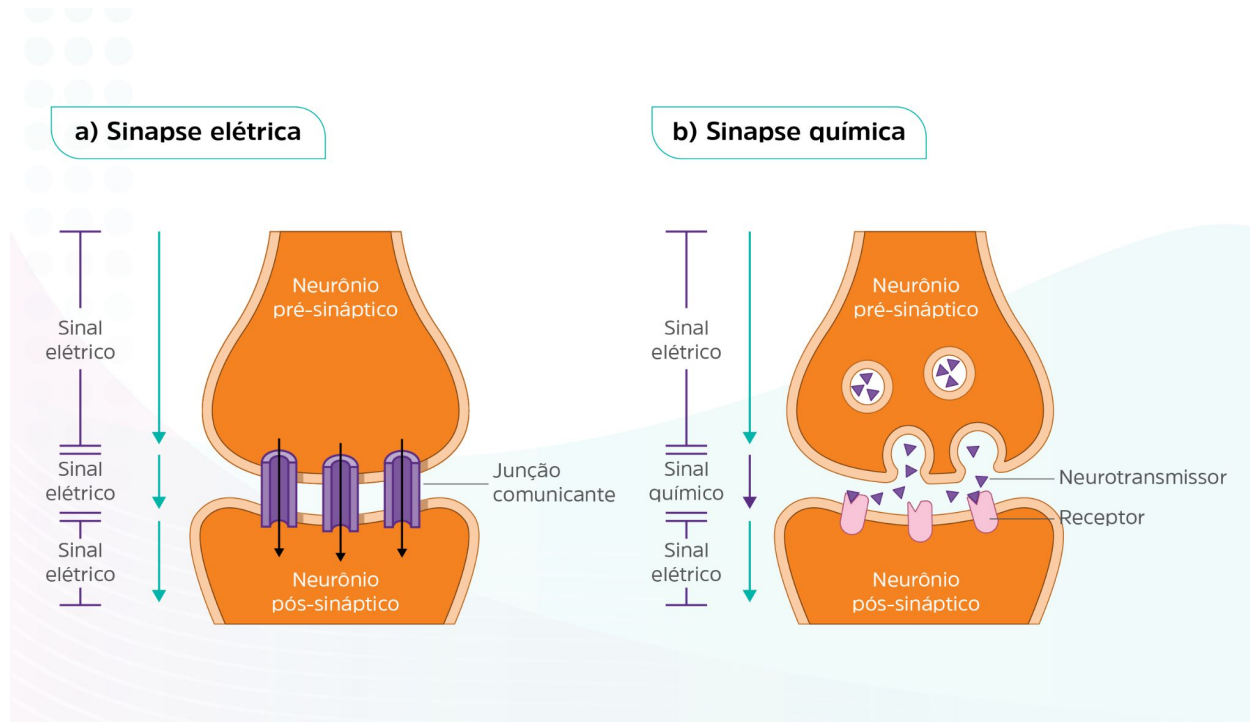


Fonte: The Synapse Revealed (Johnson - 2005).



Fonte: SILVERTHORN (2010)

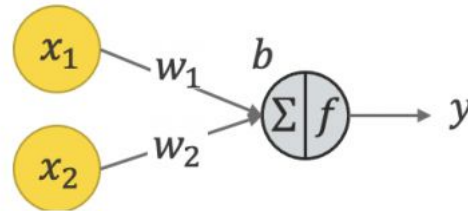
Inspiração biológica: O cérebro se organiza em redes



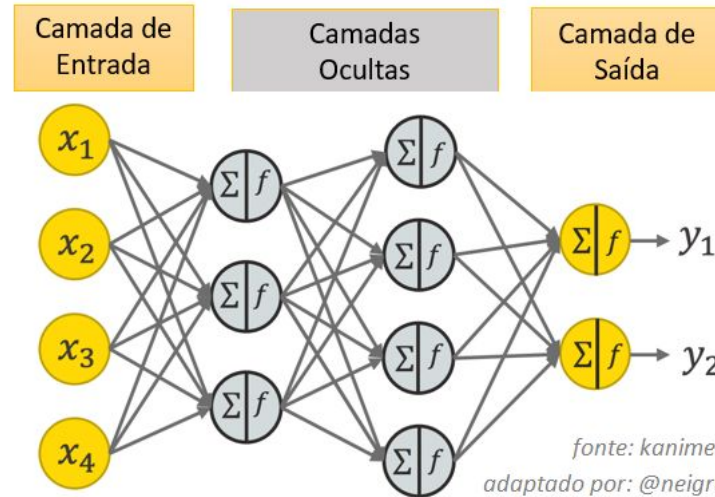
Fonte: Meu Cosmos (2015).

Redes Neurais Artificiais

Um Neurônio Artificial



Exemplo de Rede Neural Artificial



fonte: kanime.com
adaptado por: @neigrando

Quais as relações entre IA, Machine Learning e Deep Learning?

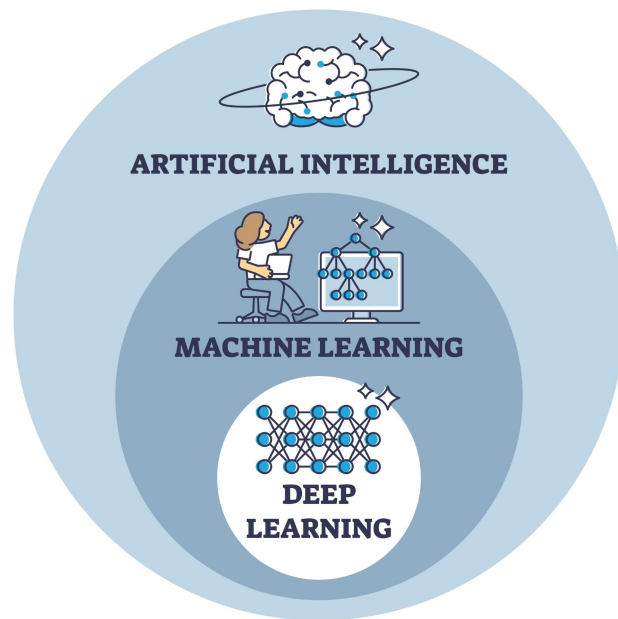


Imagem gerada por IA

Inteligência Artificial (IA)

- Um campo amplo que busca simular inteligência humana em máquinas.

Quaisquer técnica (ferramenta) que permita que os computadores mimetizem (simulem) a “inteligência” humana, usando lógica, modelos de linguagem, KNN (vizinhos mais próximos), árvores de decisão, NLP (Natural Language Processing), aprendizado de máquina (incluindo aprendizado profundo) entre outras.

Processamento de Linguagem Natural

Robótica

Reconhecimento de fala



Visão computacional

Machine Learning

- Um subcampo da IA focado em algoritmos que aprendem padrões a partir de dados, sem serem explicitamente programados para cada tarefa.

Subconjunto da grande área de IA que inclui técnicas de estatística, modelos matemáticos e algoritmos que capacitam máquinas a melhorar tarefas com base em experiência (dados).

Aprendizado Supervisionado

Aprendizado Não Supervisionado

Aprendizado por Reforço

Machine Learning - Aprendizado Supervisionado

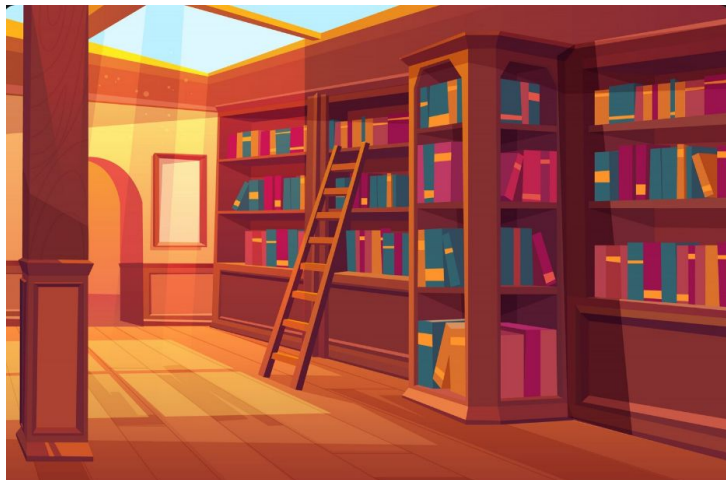
O modelo é treinado com dados rotulados (sabemos a resposta correta). Objetivando mapear uma entrada para uma saída com base em exemplos históricos.

- Solucionamos problemas de **Regressão** (previsão de preços) e **Classificação** (filtros de spam).

Machine Learning - Aprendizado Não Supervisionado

O modelo analisa dados sem rótulos ou respostas pré-definidas. Objetivando encontrar estruturas, grupos ou relações intrínsecas nos dados.

- Segmentação de clientes por comportamento e sistemas de recomendação.



Biblioteca interior, Freepik.

Deep Learning

- Uma evolução do ML baseada em redes neurais profundas, ideal para dados não estruturados (imagens, áudio).

Subconjunto do aprendizado de máquina composto de algoritmos que permitem ao software identificar padrões ocultos e assim treinar a si mesmo para realizar tarefas, como reconhecimento de voz e imagem, expondo uma grande quantidade de dados à multicamadas de redes neurais.

Redes Neurais Recorrentes

Redes Neurais Convolucionais

Redes Neurais Artificiais

Quais as relações entre IA, Machine Learning e Deep Learning?

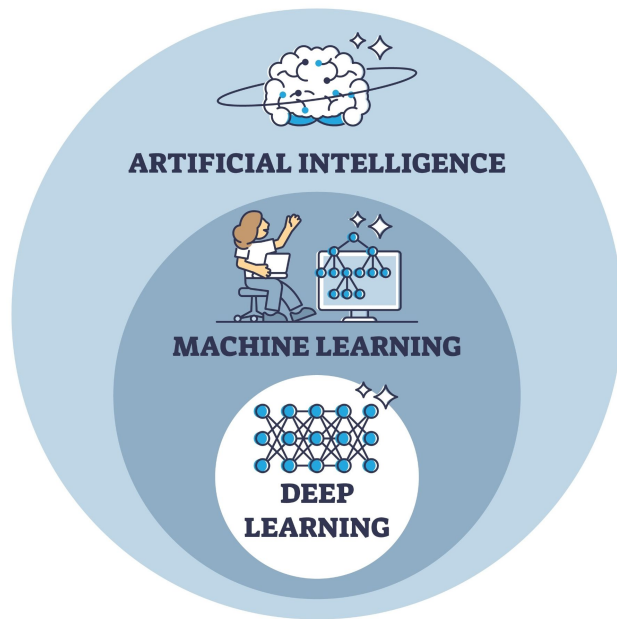
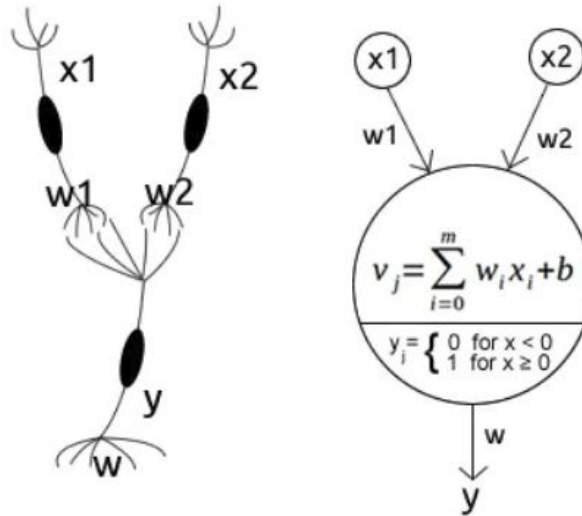


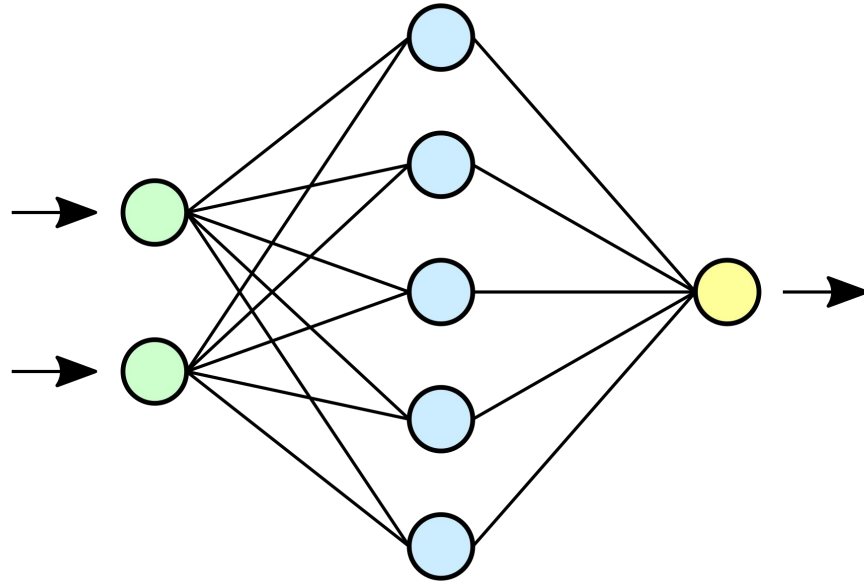
Imagem gerada por IA

Voltando ao modelo artificial



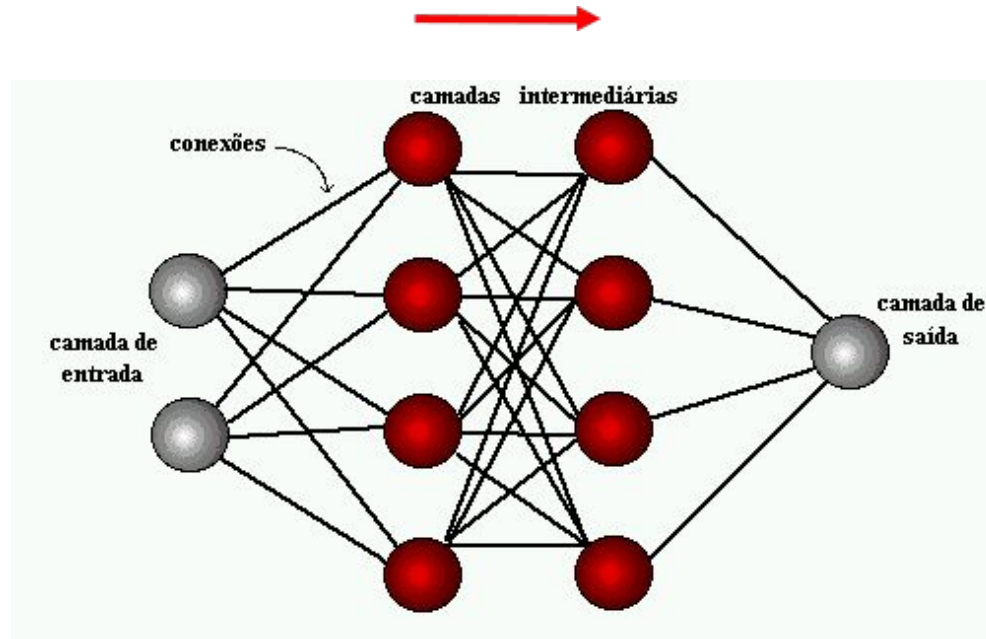
<https://medium.com/brasil-ai/entendendo-o-funcionamento-de-uma-rede-neural-artificial-4463fcf44dd0>

Voltando ao modelo artificial



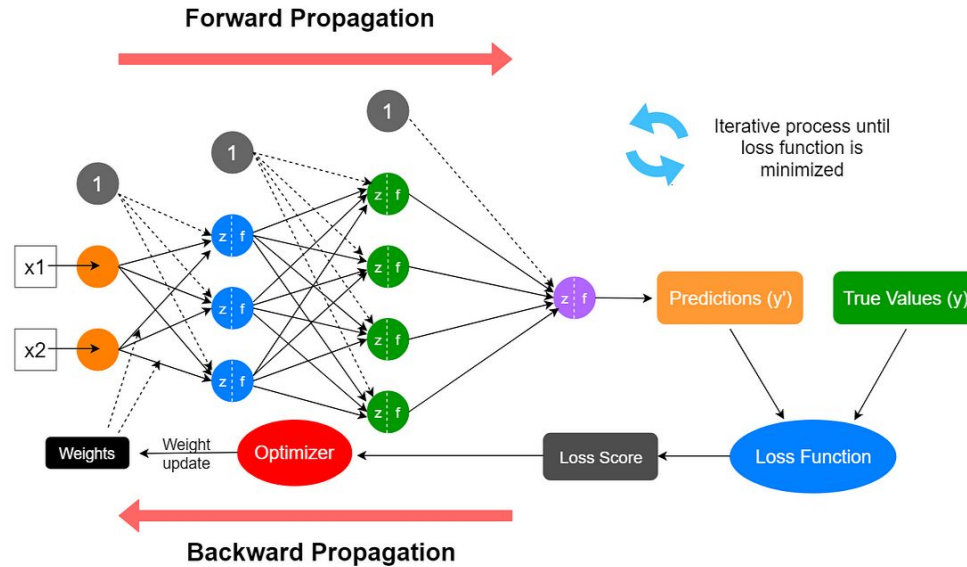
https://pt.wikipedia.org/wiki/Rede_neural_artificial

Feedforward (alimentando a rede)



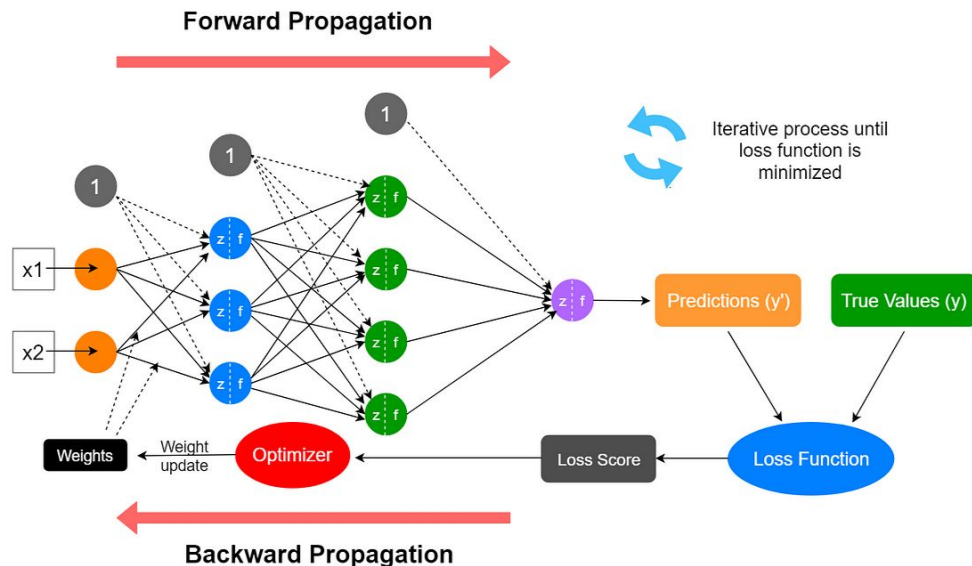
<https://sites.icmc.usp.br/andre/research/neural/>

O processo de ajuste dos pesos (Backpropagation)



<https://www.linkedin.com/pulse/understanding-forward-backward-propagation-neural-suresh-beekhane-e0rkf/>

Quando parar?



Basicamente podemos ter critérios para parar o fluxo:

- **Épocas** (estipulamos quantas vezes esse processo deve acontecer).
- **Tolerância** (definimos, por exemplo uma métrica como acurácia e deixamos o modelo chegar onde queremos).
- *Note que podemos usar as duas condições (com um OR).*

<https://www.linkedin.com/pulse/understanding-forward-backward-propagation-neural-suresh-beekhane-e0rkf/>

1. ENTRADAS

(valores dos atributos)

x_1
(idade)

x_2
(nº de estudos)

x_3
(horas de sono)



Cada entrada representa uma característica do exemplo.

2. PESOS

(força de cada conexão)

$w_1 = 0.5$

$w_2 = 2$

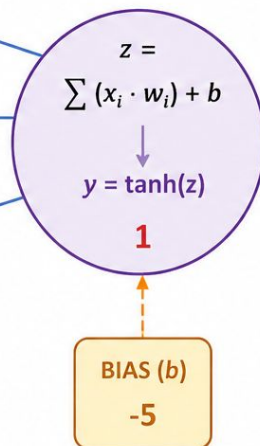
$w_3 = 1.3$

Cada peso define a importância da entrada na decisão do neurônio.

3. NEURÔNIO (SAÍDA)

(combinação + ativação)

O neurônio soma as entradas ponderadas e aplica uma função de ativação (tanh).



O bias (b) ajusta o limiar da função de ativação.

PASSO A PASSO

- 1 Entradas** (valores do exemplo):
 $x_1 = 30$, $x_2 = 4$, $x_3 = -7$
- 2 Multiplica por cada peso:**
 $30 \times 0.5 = 15$
 $4 \times 2 = 8$
 $-7 \times 1.3 = -9.1$
- 3 Soma tudo e adiciona o bias:**
 $z = 15 + 8 - 9.1 + (-5)$
 $z = 8.9$
- 4 Aplica a função de ativação (tanh):**
 $y = \tanh(8.9) \approx 1$
- 5 Saída do neurônio:**
 $y \approx 1$ (valor entre -1 e 1)



A função tanh “comprime” o resultado para ficar entre -1 e 1. Valores positivos grandes viram saídas próximas de 1.

Várias arquiteturas

- Célula de entrada
- Célula de entrada retroalimentada
- △ Célula de entrada com ruído
- Célula escondida
- Célula escondida probabilística
- △ Célula escondida com disparo
- Célula cápsula
- Célula de saída
- Célula de saída igual à de entrada
- Célula recorrente
- Célula de memória
- △ Célula de memória com portão
- Kernel
- Convolução ou pool

Um quadro quase completo de Redes Neurais

©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org

Perceptron (P)



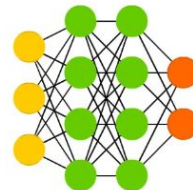
Feed Forward (FF)



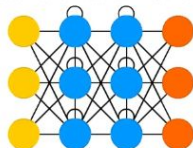
Radial Basis Network (RBF)



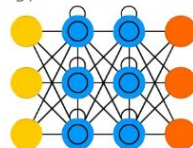
Deep Feed Forward (DFF)



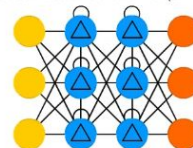
Recurrent Neural Network (RNN)



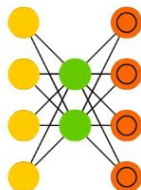
Long / Short Term Memory (LSTM)



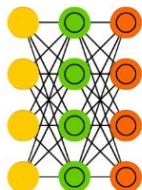
Gated Recurrent Unit (GRU)



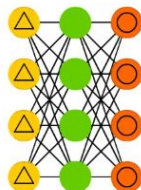
Auto Encoder (AE)



Variational AE (VAE)



Denoising AE (DAE)



Sparse AE (SAE)



<https://www.asimovinstitute.org/neural-network-zoo/>

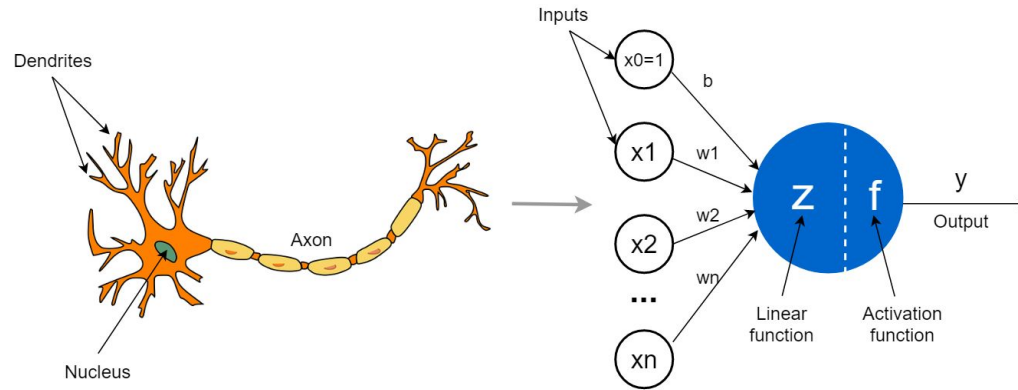
Perceptron (1958)

- **Criador:** Frank Rosenblatt (inspirado em McCulloch e Pitts).
- **O que é:** Um dos modelos mais antigos de Rede Neural Artificial.
- **Estrutura:** Composto por um **único neurônio**.
- **Objetivo:** Classificação de dados de forma **linear**.

Como Funciona o Neurônio?

- **Entradas:** Recebe os dados de entrada.
- **Pesos:** Cada entrada é multiplicada por um peso sináptico (w).
- **Soma:** Os valores são somados, gerando o potencial de ativação:
- **Ativação:** O resultado passa por uma função de ativação.
- **Saída:** Se a soma (x_i) superar o limite da função, o neurônio é ativado e gera a saída.

Perceptron



Entradas (x): Os dados iniciais que chegam ao neurônio.

Pesos (w): Definem a importância relativa de cada entrada.

Viés (b): Ajuste fixo que permite flexibilidade na decisão.

Ativação: Filtro que decide o sinal final do neurônio.

Saída (y): A decisão ou predição gerada.

Perceptron



1. As Entradas

Analizamos os fatores do dia a dia: O ingresso é barato? Vai chover? Meus amigos vão?



2. Pesos & Viés

Atribuímos importância a cada fator (peso) e consideramos nossa vontade prévia (viés).

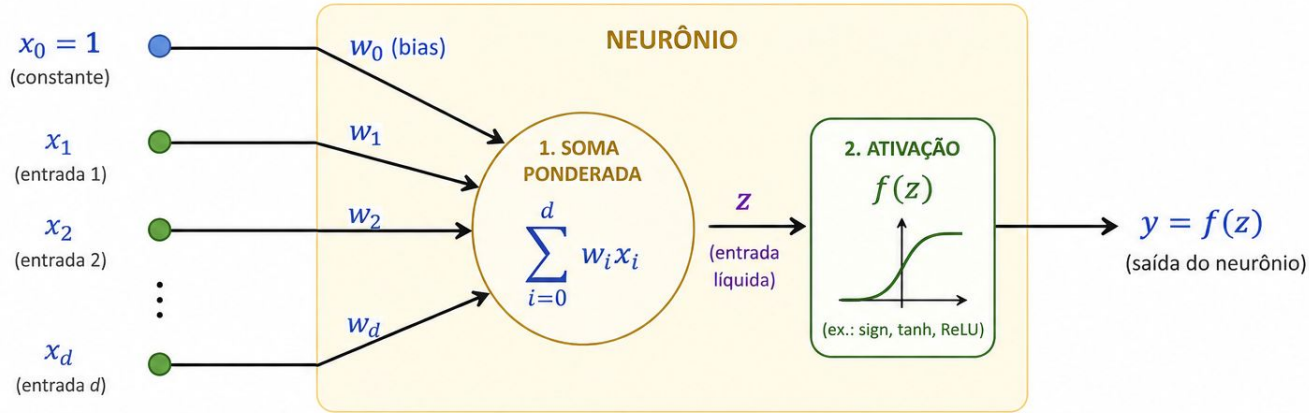


3. A Decisão

Se a soma ponderada ultrapassa um limite (ativação), decidimos: **1 (Ir ao show)** ou **0 (Ficar em casa)**.

UM NEURÔNIO (PERCEPTRON)

Combina várias entradas ponderadas, soma tudo e aplica uma função de ativação.



1 ENTRADAS E PESOS

Cada entrada x_i é multiplicada pelo seu peso w_i .

- $x_0 = 1$ é a entrada de *bias* (constante).
- O *bias* permite ajustar o limiar da ativação.

2 SOMA PONDERADA

O neurônio soma todas as entradas ponderadas:

$$z = \sum_{i=0}^d w_i x_i$$

z é a entrada líquida do neurônio.

3 ATIVAÇÃO

Aplica uma função de ativação $f(z)$ para produzir a saída.

$$y = f(z)$$

A função pode ser, por exemplo: sign, tanh, ReLU, sigmoid, etc.

4 SAÍDA

O valor y é a saída do neurônio e será usado como entrada para outros neurônios na rede.

$$y = f\left(\sum_{i=0}^d w_i x_i\right)$$

LEGENDA

● Entradas

w_i Pesos

z Entrada líquida (soma ponderada)

$f(z)$ Função de ativação

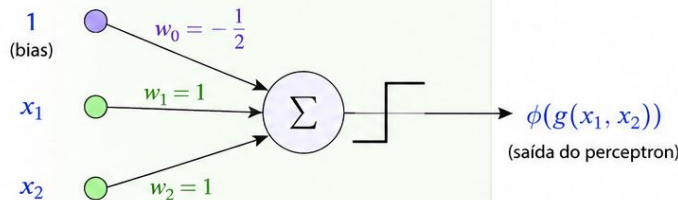
► Implementação do OR com Perceptron

O perceptron aprende a função **OR** separando as entradas 0 e 1 por uma fronteira linear.

1. TABELA-VERDADE DO OR

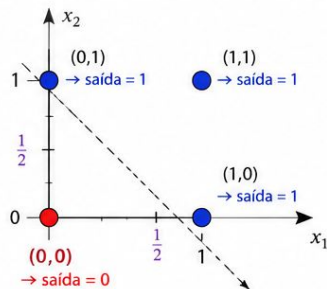
x_1	x_2	Saída desejada $\phi(g(x_1, x_2))$
0	0	0
0	1	1
1	0	1
1	1	1

2. PERCEPTRON USADO PARA IMPLEMENTAR O OR



5. FRONTEIRA DE DECISÃO

A fronteira é dada por $g(x_1, x_2) = 0 \Rightarrow x_1 + x_2 - \frac{1}{2} = 0 \Rightarrow x_2 = -x_1 + \frac{1}{2}$



--- $g(x_1, x_2) = 0$
 $x_1 + x_2 = \frac{1}{2}$
 ● Região $g \geq 0$
 (saída = 1)
 ● Região $g < 0$
 (saída = 0)

Acima da linha ($g \geq 0$):
o perceptron ativa → saída 1

Abaixo da linha ($g < 0$):
o perceptron não ativa → saída 0

3. FUNÇÃO LINEAR (ENTRADA LÍQUIDA)

$$g(x_1, x_2) = w_0 \cdot 1 + w_1 \cdot x_1 + w_2 \cdot x_2 = x_1 + x_2 - \frac{1}{2}$$

O bias ($w_0 = -\frac{1}{2}$) desloca o limiar de decisão.

4. FUNÇÃO DE ATIVAÇÃO (FUNÇÃO DEGRAU)

A função degrau transforma $g(x_1, x_2)$ na saída binária:

$$\phi(g) = \begin{cases} 1, & \text{se } g(x_1, x_2) \geq 0 \\ 0, & \text{se } g(x_1, x_2) < 0 \end{cases}$$



Assim, o perceptron produz 1 sempre que pelo menos uma entrada for 1, e 0 apenas quando ambas forem 0 → exatamente o **OR**.

Onde o Perceptron falha!

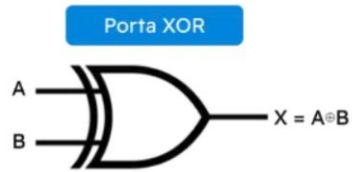


Tabela da Verdade		
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

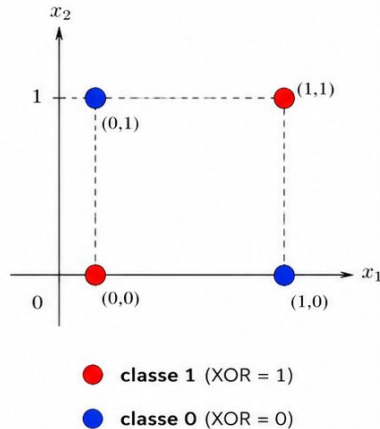
https://www.makerhero.com/blog/o-que-sao-portas-logicas/?srsltid=AfmBOopevld_1X55_usMZOM0zOJhXT_QrQK3g-md-tZpTocbU6F8eVN_

Perceptron falha no XOR ✗



O perceptron é um classificador linear:
não consegue separar linearmente os pontos do XOR.

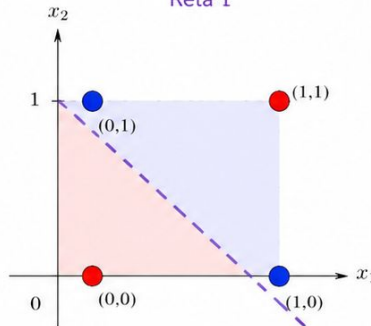
Problema: XOR



No XOR, os pontos da mesma classe não são linearmente separáveis.

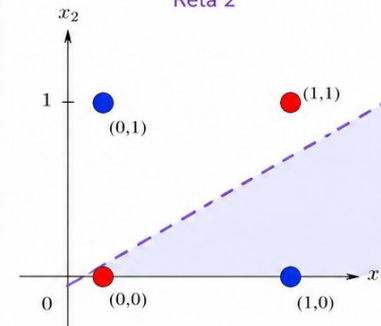
Tentativas de separação linear (retas)

Reta 1



✗ Classifica (0,1) corretamente (0) e (1,0) corretamente (0), mas **erra (0,0) que é 1.**

Reta 2



✗ Classifica (0,0) corretamente (1) e (1,1) corretamente (1), mas **erra (0,1) que é 0.**



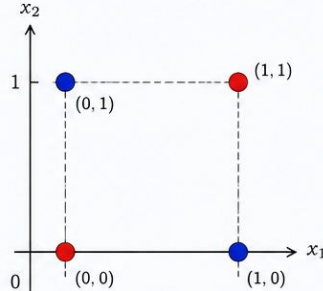
Não existe nenhuma reta que separe corretamente as duas classes do XOR. Logo, um perceptron (classificador linear) sempre falhará nesse problema.



Contornando o problema do XOR com mais de um perceptron ✓

Usamos dois perceptrons na camada oculta para criar fronteiras lineares intermediárias, que permitem que um terceiro perceptron na saída resolva o XOR.

Problema: XOR

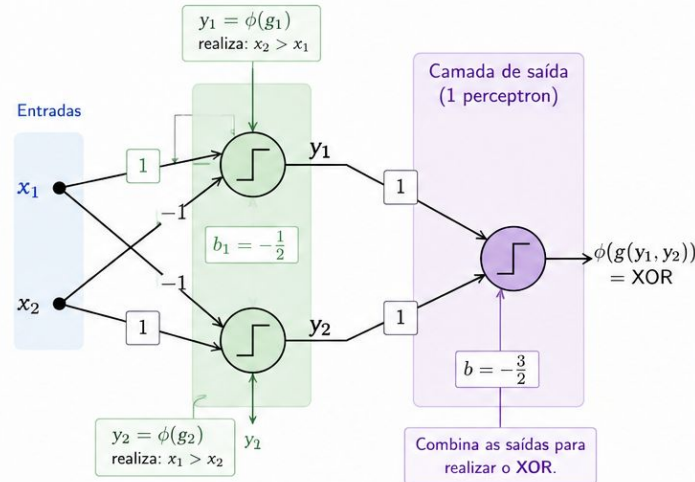


● classe 1 (XOR = 1)

● classe 0 (XOR = 0)

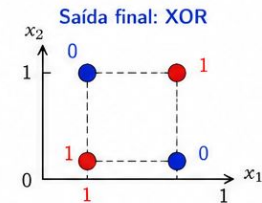
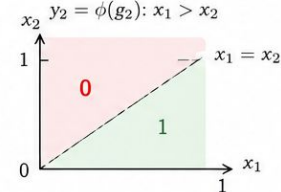
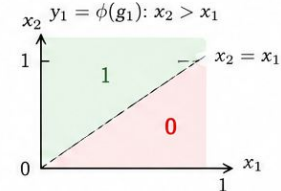
✗ Nenhuma reta única consegue separar corretamente as classes.

Solução: Rede com 2 perceptrons na camada oculta + 1 na saída



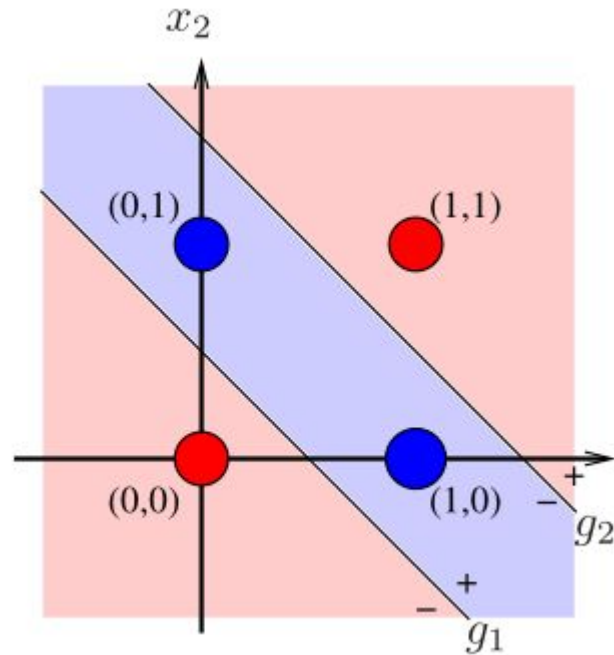
x_1	x_2	$y_1 = \phi(g_1)$	$y_2 = \phi(g_2)$	$\phi(g(y_1, y_2))$ (saída)	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	0	1	1	1
1	1	0	0	0	0

Fronteiras aprendidas



✓ Com duas fronteiras lineares intermediárias, o último perceptron consegue resolver o XOR!

Aumentar a rede resolve o problema



Limites do ML Clássico



Engenharia Manual

Algoritmos tradicionais exigem que humanos extraiam e escolham manualmente cada característica relevante dos dados.



Dados Não Estruturados

Modelos como KNN ou Árvores sofrem para processar imagens, vídeos, áudio e linguagem natural diretamente.



Teto de Desempenho

Mesmo recebendo milhões de dados adicionais, o desempenho de modelos clássicos tende a estagnar em um platô.

Desempenho com Dados Massivos

Deep Learning (Big Data) 95% Acurácia

Random Forest / XGBoost 78% Acurácia

Regressão Logística / KNN 62% Acurácia

As redes neurais continuam melhorando à medida que recebem volumes massivos de dados, superando o teto de desempenho dos modelos tradicionais.

ML Clássico vs. Redes Neurais

Como escolher a abordagem ideal para o seu problema de negócios com base em dados, custos e requisitos de explicabilidade.

A Escolha do Modelo Certo

Machine Learning Clássico

Algoritmos baseados em estatística e matemática estruturada (como XGBoost, Random Forest e Regressões).

Foco: Tabelas, dados rotulados, alta velocidade e facilidade de auditoria.

Redes Neurais (Deep Learning)

Modelos com arquiteturas em camadas inspiradas na percepção biológica.

Foco: Dados não estruturados (visão, texto, áudio), abstração complexa e escala massiva.

Pilares de Decisão

Algumas dimensões essenciais determinam se o projeto deve usar modelos tradicionais ou redes profundas.

Lembrando que, como eu costumo dizer, nada está escrito em pedra. E, às vezes, dependendo do problema, a gente flutua entre uma possibilidade e outra.

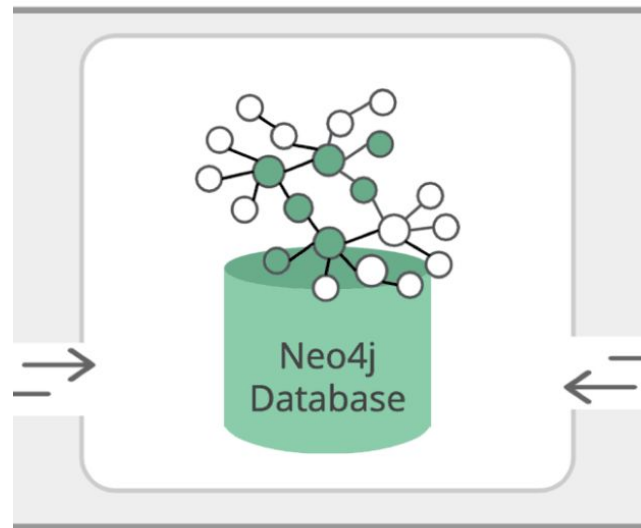
Quando Usar ML Clássico

Dados Estruturados & Negócios

O Machine Learning Clássico é imbatível quando lidamos com dados tabulares, planilhas e bancos de dados relacionais.

Ideal quando o volume de dados é pequeno a médio (até dezenas de milhares de linhas) e exige alta explicabilidade para auditoria.

Native Graph
Creation & Persistence



Source: www.movagain.dk

Quando Usar Redes Neurais

Dados Não Estruturados & Escala

Redes Neurais brilham ao processar imagens, vídeos, áudios, textos livres e séries temporais de alta complexidade.

Indispensáveis quando o projeto possui volume massivo de dados (Big Data) e a precisão bruta é prioridade sobre a explicabilidade.



Source: www.vecteezy.com

Desempenho vs Volume de Dados



Com volumes menores de dados, modelos clássicos superam redes neurais. Com dados massivos, o Deep Learning expande a acurácia sem atingir platô precocemente.

Engenharia de Atributos



ML Clássico: Manual

Cientistas de dados selecionam e transformam atributos manualmente (ex: médias, agregações, filtros de imagem).



Deep Learning: Automático

Camadas profundas aprendem hierarquias de representação diretamente das entradas brutas sem intervenção humana.



O Impacto Prático

ML Clássico exige mais conhecimento do negócio; Redes Neurais exigem mais dados brutos e poder computacional.

Transparência de Modelos

Explicabilidade x Caixa Preta

Em setores altamente regulados (bancos, seguros, saúde e área jurídica), responder por que uma decisão foi tomada é um requisito legal.

Algoritmos de ML Clássico mostram a importância clara de cada variável. Redes Neurais operam com milhões de parâmetros interligados, agindo como caixas pretas.

100%

Auditabilidade Exigida

Matriz Comparativa de Custos (só uma visão geral)

Dimensão	Machine Learning Clássico	Redes Neurais (Deep Learning)
Hardware Necessário	CPUs Padrão (Laptops / Servidores)	GPUs / TPUs de Alto Desempenho
Tempo de Treinamento	Segundos a Minutos	Horas, Dias ou Semanas
Necessidade de Dados	Baixa a Média (1k – 100k linhas)	Alta a Massiva (100k+ imagens/textos)
Facilidade de Implementação	Alta (Rápida iteração de protótipos)	Média a Baixa (Pipeline e MLOps complexos)

Casos de Uso no Mundo Real

Machine Learning Clássico

- **Finanças:** Score de Crédito e Fraude Tabular.
- **Marketing:** Previsão de Churn e Propensão de Compra.
- **Vendas:** Precificação Dinâmica de Imóveis e Produtos.
- **Operações:** Manutenção Preditiva com Sensores.

Redes Neurais (Deep Learning)

- **Visão Computacional:** Exames Médicos por Imagem.
- **NLP:** Chatbots, LLMs e Análise de Sentimento.
- **Automação:** Veículos Autônomos e Robótica.
- **Áudio:** Reconhecimento e Síntese de Voz.

Framework de Decisão



Regra de Ouro na Escolha

Comece sempre com o modelo mais simples e explicável que resolva satisfatoriamente o problema de negócio!

Lembrando que, como eu costumo dizer, nada está escrito em pedra. E, às vezes, dependendo do problema, a gente flutua entre uma possibilidade e outra.



Modelo Preditivo

MÓDULO 2 - SEMANA 5 - AULA 2



SCITI.SC

AGENDA

- TensorFlow
- PyTorch
- Keras
- O que são séries temporais?
- Exemplos práticos

Pilares importantes quando se fala em IA



TensorFlow



PyTorch



Keras

Embora frequentemente citados juntos, cada framework cumpre um papel distinto no ecossistema de aprendizado em IA.

Entendendo os Conceitos

Motores de Cálculo (Engines): *TensorFlow & PyTorch*: São plataformas completas de baixo/médio nível. Gerenciam grafos computacionais, aceleração em GPUs/TPUs e cálculo automático de gradientes.

Entendendo os Conceitos

Interface de Alto Nível (API): *Keras*: É uma camada de abstração projetada para humanos. Ela simplifica a criação de redes neurais, podendo rodar por cima do TensorFlow, PyTorch ou JAX.

TensorFlow



TensorFlow

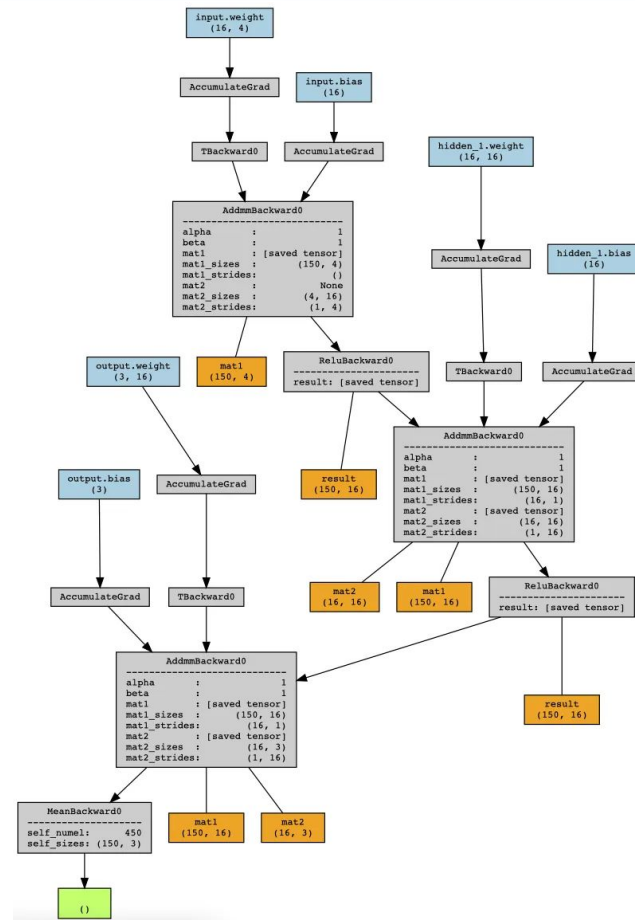
O Ecossistema do Google: Criado pelo Google em 2015, o TensorFlow foi construído focado em escala industrial, robustez e implantação corporativa.

Possui o ecossistema de deploy mais maduro do mercado, oferecendo suporte nativo para nuvem, servidores (TF Serving), navegadores (TF.js) e dispositivos móveis (TF Lite).

PyTorch: Foco em Pesquisa

O Motor da Meta & Academia: Lançado pela Meta (Facebook) em 2016, o PyTorch conquistou pesquisadores devido ao seu estilo nativamente Pythonico e grafos dinâmicos.

É o padrão dominante no mundo acadêmico e no desenvolvimento dos mais modernos LLMs e Transformers (como os modelos da Hugging Face e ChatGPT).



Keras: Consegue simplificar as coisas

Prototipagem Rápida: Permite criar, compilar e treinar modelos complexos de redes neurais com pouquíssimas linhas de código limpo.

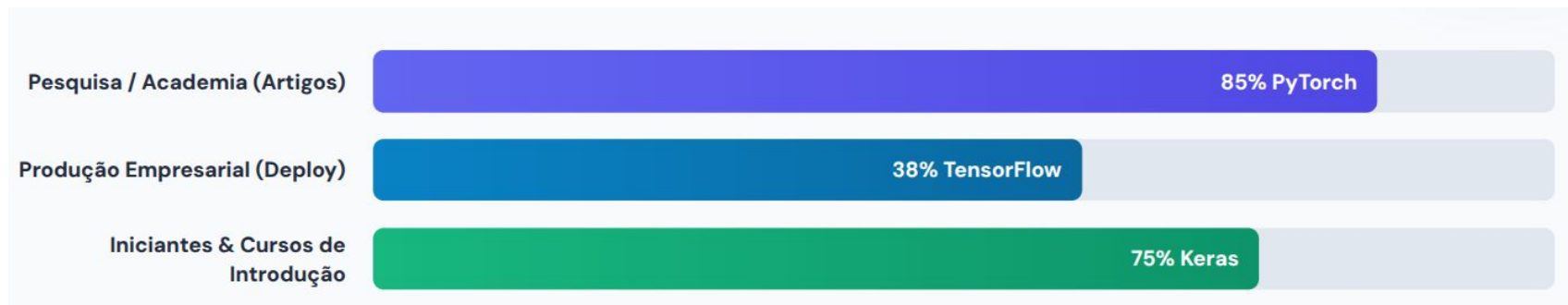
Multi-Backend (Keras 3): Com o Keras 3, um mesmo modelo pode rodar sobre TensorFlow, PyTorch ou JAX sem alterar o código-fonte.

Amigável a Iniciantes: Reduz a curva de aprendizado inicial, abstraindo a matemática pesada e focando na arquitetura da rede.

Comparativo Direto

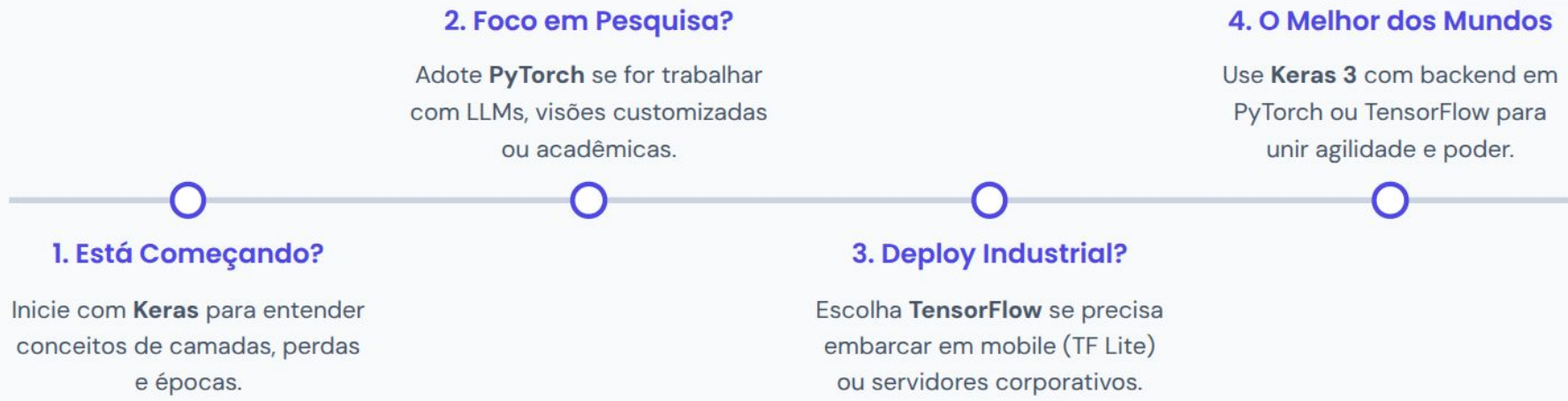
Característica	TensorFlow	PyTorch	Keras
Criador / Mantenedor	Google	Meta (Facebook)	François Chollet / Google
Nível de Abstração	Médio / Baixo	Médio / Baixo	Alto (Simplicidade)
Curva de Aprendizado	Moderada a Alta	Moderada	Muito Baixa (Fácil)
Foco Principal	Produção & Escala	Pesquisa & Flexibilidade	Prototipagem & Produtividade

Domínio por Área de Aplicação



O PyTorch lidera disparado em pesquisas científicas e modelos genAI, enquanto o TensorFlow mantém forte presença em infraestruturas legadas e mobile.

Como Escolher Seu Framework



Não existe biblioteca "melhor", existe a ferramenta certa para cada objetivo do seu projeto.

De Scikit-Learn para Keras

Scikit-Learn (ML Clássico)

Foco: Algoritmos prontos em bloco único (Random Forest, SVM, Regressão).

Interface: Métodos diretos como `fit(X, y)` e `predict(X)`.

Limitação: Arquitetura fixa; não permite desenhar a estrutura interna de neurônios.

TensorFlow + Keras (Deep Learning)

Foco: Arquiteturas modulares e altamente personalizáveis.

Interface: Montagem camada por camada com controle fino do aprendizado.

Vantagem: Aceleração via GPU/TPU e suporte a volumes massivos de dados.

Tensors: As Estruturas de Dados



1. Escalares & Vetores

0D / 1D Tensors: Um único número (rank 0) ou um vetor unidimensional de dados (rank 1), similar a uma lista em Python.



2. Matrizes (2D)

2D Tensors: Estrutura em linhas e colunas. É o formato padrão para tabelas e datasets estruturados do Scikit-Learn.



3. Tensores Multidimensionais

3D+ Tensors: Matrizes empilhadas. Usados para imagens (Altura, Largura, Cores) e lotes de dados (Batches).

O Ciclo de Vida de um Modelo



As Camadas: Sequential e Dense

tf.keras.models.Sequential

É o container principal do Keras. Permite empilhar camadas em uma sequência simples, onde a saída da camada N conecta automaticamente com a entrada da camada N+1.

tf.keras.layers.Dense

A camada tradicional totalmente conectada (*Fully Connected*). Cada neurônio recebe dados de todos os neurônios da camada anterior, calculando $z = w * x + b$.

Funções de Ativação Básicas

ReLU (Camadas Ocultas)

activation='relu' ==> Converte valores negativos em 0 e mantém positivos. Evita o desaparecimento do gradiente durante o treino.

Sigmoid (Saída Binária)

activation='sigmoid' ==> Comprime a saída entre 0 e 1. Ideal para problemas de classificação de duas classes (0 ou 1).

Softmax (Multiclasse)

activation='softmax' ==> Gera uma distribuição de probabilidades onde a soma de todas as classes é igual a 1 (100%).

Compilação e Otimização

Loss Function (Perda): Mede o quão "errado" o modelo está (ex: `binary_crossentropy`, `categorical_crossentropy`).

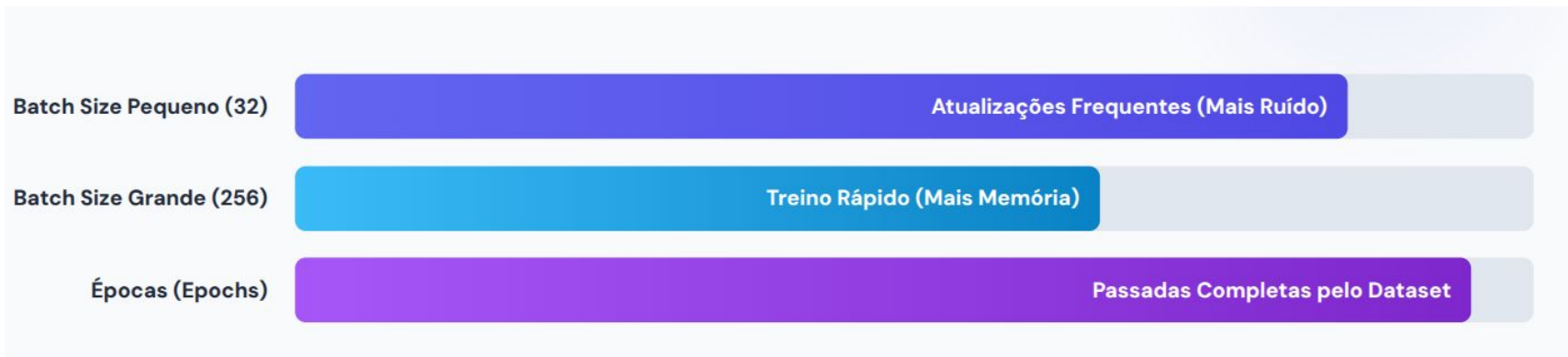
Optimizer (Otimizador): Algoritmo que ajusta os pesos (ex: Adam, SGD).

Metrics (Métricas): Indicadores legíveis acompanhados durante o treino (ex: 'accuracy').

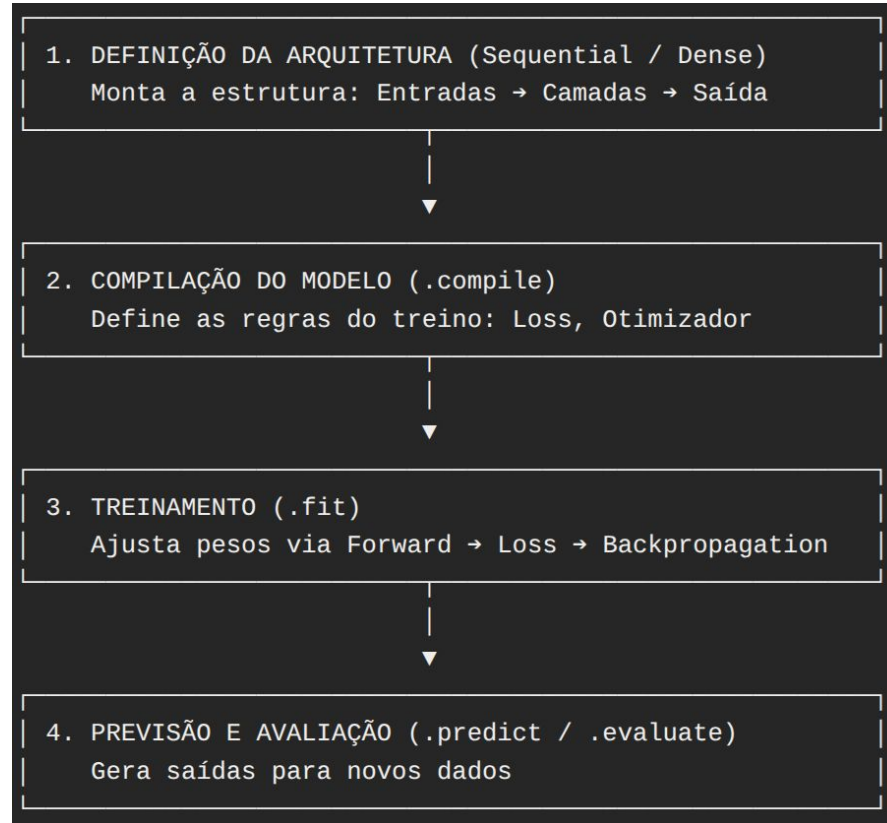
Épocas e Batch Size

Epoch: Quantas vezes o modelo verá todo o dataset.

Batch Size: O número de amostras processadas antes de atualizar os pesos.



Proposta de Fluxo para uma Rede Neural TensorFlow/Keras





FLUXO DO TENSORFLOW (Keras)

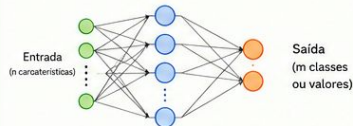
As 4 etapas principais para criar, treinar e usar modelos de Machine Learning

1

DEFINIÇÃO DA ARQUITETURA (Sequential / Dense)

Monte a estrutura da rede neural: entradas, camadas ocultas e saída.

EXEMPLO DE ARQUITETURA



```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(n,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(m, activation='softmax')
])
```



O que acontece aqui?

- ✓ Você define quantas camadas e neurônios terá.
- ✓ Escolhe a função de ativação (ReLU, Sigmoid, Softmax, etc.).

2

COMPILAÇÃO DO MODELO (.compile)

Define as regras do treino: função de perda (loss), otimizador e métricas.

REGRAS DO TREINO



Loss
Como medir o erro



Otimizador
Como atualizar os pesos



Métricas
Como avaliar o desempenho

```
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```



O que acontece aqui?

- ✓ Loss: diz ao modelo o que é "erro".
- ✓ Otimizador: define como o modelo vai aprender (ex: Adam, SGD).
- ✓ Métricas: o que vamos observar (accuracy, precisão, etc.).

3

TREINAMENTO (.fit)

O modelo aprende com os dados: faz previsões, calcula o erro e ajusta os pesos (iterativamente).

COMO ACONTECE O TREINO



Dados de Treino



Forward (Predição)



Calcula Loss (Erro)



Backpropagation (Ajusta pesos)

```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```



O que acontece aqui?

- ✓ O modelo vê os dados em pequenos lotes (batches).
- ✓ Calcula o erro e ajusta os pesos usando backpropagation.
- ✓ Repetimos por várias épocas até o modelo aprender.

4

PREVISÃO E AVALIAÇÃO (.predict / .evaluate)

Use o modelo treinado para gerar previsões e avaliar seu desempenho em novos dados.

USO DO MODELO



Novos dados



Modelo treinado



Previsões (saídas)



Avaliação do desempenho

```
# Fazer previsões
y_pred = model.predict(X_test)

# Avaliar o modelo
loss, acc = model.evaluate(X_test, y_test)
print(f"Acurácia: {acc:.2%}")
```



O que acontece aqui?

- ✓ .predict() gera as saídas do modelo para novos dados.
- ✓ .evaluate() calcula o desempenho (usando os dados de teste).

RESUMO RÁPIDO DO FLUXO



1. Definir arquitetura
Como será o modelo.



2. Compilar
Quais são as regras do aprendizado.



3. Treinar
O modelo aprende com os dados.



4. Prever e avaliar
Usamos o modelo e checamos seu desempenho.



TensorFlow

Abstrai o trabalho pesado para você focar no que importa: resolver problemas com dados!

O que é uma Série Temporal?

Dados em Sequência no Tempo

Uma série temporal é simplesmente uma coleção de informações organizadas em ordem cronológica (horas, dias, meses ou anos).

- **Tendência:** A direção geral no longo prazo (ex: o crescimento contínuo da população ou o aumento de vendas).
- **Sazonalidade:** Padrões que se repetem em ciclos fixos (ex: venda de sorvete no verão ou aumento de vendas no Natal).



Por que os Métodos Tradicionais Falham?

Matemática Rígida (A Linha Reta)

- **Aproximação Simples:** Modelos tradicionais tentam desenhar linhas retas ou fórmulas pré-fixadas.
- **Falta de Flexibilidade:** Funcionam bem para coisas previsíveis e calmas, mas quebram com mudanças bruscas.

O Mundo Real é Caótico

- **Eventos Inesperados:** Promoções relâmpago, mudanças no clima, pandemias ou tendências de redes sociais.
- **Relações Complexas:** Nem tudo no mundo real segue uma regra matemática perfeita. O comportamento humano varia!

A Solução: Redes Neurais & Memória

Um Cérebro que Lembra do Passado

Aprender Sequências: Em vez de olhar apenas para o número de hoje, a rede neural analisa a sequência inteira dos dias anteriores.

Intuição da "Memória": O algoritmo consegue “lembrar” do que aconteceu há 5 minutos ou há 6 meses para entender o contexto atual.

Adaptação Contínua: Descobre padrões escondidos sem precisarmos ditar regras rígidas.

Séries Temporais no Dia a Dia



Streaming & Mídia

Netflix & Spotify: Preveem em qual horário você costuma assistir séries para sugerir o conteúdo certo no momento exato.



Mobilidade Urbana

Uber & 99: Ajustam a tarifa dinâmica e posicionam motoristas prevendo o aumento de passageiros em saídas de shows.



Saúde & Logística

Hospitais & Mercados: Preveem a ocupação de leitos em épocas de gripe e o estoque ideal para evitar desperdício de alimentos.

Algumas abordagens:

- Redes Neurais Recorrentes (RNN) e suas variantes (LSTM / GRU)
- CNNs 1D (Redes Neurais Convolucionais Unidimensionais)
- Transformers (Que vem sendo explorado para resolver vários tipos de problemas)
- Abordagens matemáticas/estatísticas clássicas



Modelo Preditivo

MÓDULO 2 - SEMANA 5 - AULA 3



SCITI.SC

AGENDA

- Exemplos de perceptron
- Regressão e Classificação com TensorFlow
- Atividade de reflexão - Redes Neurais
- Spoiler do Mini-Projeto

ATIVIDADE GUIADA

MACHINE LEARNING E PROGRAMAÇÃO | Atividade Prática de Implementação



Criada com Gemini

Notebooks

Posteriormente!!!

Sessão de links

- 1) [Inteligência Artificial aplicada à Bioinformática – BIOINFO](#)
- 2) <https://www.linkedin.com/pulse/ascens%C3%A3o-das-m%C3%A1quinas-pensantes-jornada-da-ia-desde-sousa-da-silva-qhxtf/>
- 3) [O que são Portas Lógicas? Tipos, tabela da verdade e exercícios - MakerHero](#)
- 4) [The Perceptron - Jonty Sinai](#)
- 5) [The Neural Network Zoo - The Asimov Institute](#)
- 6) [Redes Neurais Artificiais](#)
- 7) [Rede neural artificial – Wikipédia, a enciclopédia livre](#)
- 8) [O que é o TensorFlow?](#)
- 9) [O que é PyTorch? | IBM](#)
- 10) [Keras | TensorFlow Core](#) (documentação)
- 11) [O que são transformadores em inteligência artificial?](#)

AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique aqui ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.

